

SMC CLINT — Low-Level Design

SMC CLINT — Low-Level SystemC Implementation

Document: 02_CLINT_LowLevel_Design.md **Module:** smc::clint (peripherals/clint/) **SystemC:** 3.0.2 Accellera (compatible with $\geq 2.3.4$) **CCI:** Accellera SystemC CCI 1.0.2 (/Users/pdroy/cci) **Status:** Code-validated; matches peripherals/clint/include/clint.h and src/clint.cpp **Companion docs:** - 01_CLINT_Specification.md — externally-observable behaviour - 03_CLINT_Test_Plan.md — verification strategy & test list

Contents

1. Purpose & scope
 2. Source layout
 3. Module structure
 - 3.1 Declaration order matters
 - 3.2 clint_cfg vs CCI
 - 3.3 CCI parameter catalogue
 - 3.4 Ports
 - 3.5 Public methods (API)
 - 3.6 Internal methods and SC processes
 - 3.7 File-local helpers
 - 3.8 Function call flow
 4. TLM-2.0 interface implementation
 5. Register decode
 6. Internal data structures
 7. Process / event topology — single-driver discipline
 8. Behavioural algorithms
 9. Reset implementation
 10. Loosely-timed timing model
 11. Error handling implementation
 12. Debug & verification hooks
 13. Integration with smc_top
 14. Modeling decisions & trade-offs
 15. Performance & footprint
 16. Known limitations & future work
 17. Code walkthrough — function reference
 18. Build, packaging and dependency notes
-

1. Purpose & scope

[01_CLINT_Specification.md](#) defines **what** the CLINT looks like from the outside — sockets, signals, register layout, protocol semantics. This document defines **how** the SystemC model is implemented internally:

- the data structures it owns,
- the SystemC process topology that drives them,
- the algorithms used for MTIME ticks, MTIP comparison, and MSIP propagation,
- the design decisions taken to satisfy SystemC 3.0's strict single-driver rule, the RISC-V machine-mode timer/SW-interrupt spec, and the SMC's loosely-timed budget,
- the verification hooks exposed to the test bench, and
- the exact integration recipe required to drop the model into `smc_top` and run firmware against it.

It is the implementation contract that any future maintainer (or an alternate implementation that wants to remain bit-compatible with the SMC CLINT) should be able to read in isolation.

2. Source layout

```
peripherals/clint/
├── CMakeLists.txt
├── README.md
├── run_tests.sh           Build + run convenience script
├── doc/
│   ├── 01_CLINT_Specification.md
│   ├── 02_CLINT_LowLevel_Design.md (this file)
│   ├── 03_CLINT_Test_Plan.md
│   └── figures/           SVG diagrams + _build_svgs.py
├── include/
│   ├── clint.h            SC_MODULE(clint) declaration + cci_param
│   └── smc_tlm_extensions.h Thin shim → cpu_cluster/include/smc_axi_extension.h
│                           (+ fabric-wide source_id_t enum)
├── src/
│   └── clint.cpp           Implementation
├── test/
│   ├── CMakeLists.txt
│   ├── clint_tb.cpp       Deterministic TB (TC-1..TC-22)
│   └── clint_tick_tb.cpp  Auto-tick TB (TA-1..TA-3)
└── build/                 (out-of-source CMake build tree)
```

Total payload: ~700 lines of C++17 incl. tests, ~330 lines excl. tests.

3. Module structure

```
namespace smc {

struct clint_cfg {
    unsigned num_harts      = 4;
    double   tick_period_ns = 100.0;

    static constexpr uint64_t SMC_BASE_ADDR    = 0xC800'0000ULL;
    static constexpr uint64_t MSIP_BASE        = 0x0000;
    static constexpr uint64_t MSIP_STRIDE       = 0x4;
    static constexpr uint64_t MTIMECMP_BASE     = 0x4000;
    static constexpr uint64_t MTIMECMP_STRIDE   = 0x8;
    static constexpr uint64_t MTIME_OFFSET     = 0xBFF8;
    static constexpr uint64_t WINDOW_SIZE      = 0x10000;    // 64 KiB
};

class clint : public sc_core::sc_module {
protected:
    cci::cci_param<unsigned, cci::CCI_IMMUTABLE_PARAM> num_harts_p_;
    cci::cci_param<double,   cci::CCI_IMMUTABLE_PARAM> tick_period_ns_p_;
    cci::cci_param<double> access_delay_ns_p_;

public:
    SC_HAS_PROCESS(clint);

    tlm_utils::simple_target_socket<clint> reg_socket;
    sc_core::sc_vector<sc_core::sc_out<bool>> msip_o;    // num_harts
    sc_core::sc_vector<sc_core::sc_out<bool>> mtip_o;    // num_harts
    sc_core::sc_in<bool> rst_n_i;

    explicit clint(sc_core::sc_module_name name, clint_cfg cfg = clint_cfg{});

    // Test-bench back-door (no socket, no side effects)
    uint64_t dbg_mtime      () const;
    uint64_t dbg_mtimecmp (unsigned hart) const;
    uint32_t dbg_msip      (unsigned hart) const;
    bool     dbg_mtip      (unsigned hart) const;
    void     dump_state    (std::ostream& = std::cout) const;
    void     dbg_set_mtime(uint64_t value);

private:
    void b_transport (tlm::tlm_generic_payload& gp, sc_core::sc_time& d);
    unsigned int transport_dbg(tlm::tlm_generic_payload& gp);

    void reset_proc();
    void tick_method();
    void output_method();
    void schedule_recompute();

    bool reg_read (uint64_t off, unsigned access_size,
                  uint32_t& data_lo, uint32_t& data_hi) const;
    bool reg_write(uint64_t off, unsigned access_size,
                  uint32_t data_lo, uint32_t data_hi);

    clint_cfg cfg_;
    uint64_t mtime_ = 0;
    std::vector<uint64_t> mtimecmp_;
    std::vector<uint8_t> msip_;
    std::vector<bool> msip_cache_;
    std::vector<bool> mtip_cache_;

    sc_core::sc_event recompute_event_;
};
```

```

    sc_core::sc_event tick_event_;
    sc_core::sc_time  access_delay_;
    sc_core::sc_time  tick_period_;
};

} // namespace smc

```

The CCI parameters are **declared protected** (rather than private) so a subclass — for example a parity-checked safety-extension wrapper — can introspect them without going through the broker.

3.1 Declaration order matters

The CCI parameters are declared **before** the `sc_vector` ports so that they appear earlier in the member-initialiser list. The vectors need `num_harts_p_.get_value()` for sizing, and CCI's broker has already applied any preset by the time the param's constructor runs. Re-ordering the declarations would size the vectors from the C++ default rather than the CCI preset.

3.2 `clint_cfg` vs CCI

`clint_cfg` is the **legacy / non-CCI** entry point, kept so that the module still behaves sensibly when constructed without a CCI broker (unit tests, examples, simple integrators). It carries:

- `num_harts` — used as the **default** for the `num_harts` CCI param.
- `tick_period_ns` — used as the **default** for the `tick_period_ns` CCI param.

When a CCI preset is set before construction, the preset wins; `cfg_` is then re-synced from `*_p_.get_value()` so the rest of the constructor and all member functions see consistent values.

3.3 CCI parameter catalogue

Hierarchical name	Type	Mutability	Default	Purpose
<code>...clint.num_harts</code>	<code>cci_param<unsigned></code>	immutable	4	Per-hart MSIP / MTIMECMP / output count.
<code>... clint.tick_period_ns</code>	<code>cci_param<double></code>	immutable	100.0	MTIME tick period (ns). 0 disables.
<code>... clint.access_delay_ns</code>	<code>cci_param<double></code>	mutable	2.0	TLM annotated delay (ns).

All parameters carry a free-form description string (visible via `cci_param_handle::get_description()`) and a small set of metadata key/value pairs (`add_metadata("rdl_dimension", ...)`, `add_metadata("unit", ...)`, etc.) so introspection tools can show provenance and units.

3.4 Ports

Port	Type	Dir	Purpose
<code>reg_socket</code>	<code>tlm_utils::simple_target_socket<clint></code>	target	TLM-2.0 register access (AXI4-Lite-style initiator).
<code>msip_o[h]</code>	<code>sc_core::sc_vector<sc_core::sc_out<bool>></code>	out	Machine software interrupt for hart <code>h</code> ; sized from <code>num_harts</code> .
<code>mtip_o[h]</code>	<code>sc_core::sc_vector<sc_core::sc_out<bool>></code>	out	Machine timer interrupt for hart <code>h</code> (level: <code>MTIME ≥ MTIMECMP[h]</code>).
<code>rst_n_i</code>	<code>sc_core::sc_in<bool></code>	in	Active-low synchronous reset.

3.5 Public methods (API)

Method	Signature	Purpose
constructor	<code>clint(sc_module_name, clint_cfg = {})</code>	Build, resolve CCI, size vectors/ports, register callbacks/processes (see §3 ctor + §7).
<code>dbg_mtime</code>	<code>uint64_t dbg_mtime() const</code>	Back-door read of the global MTIME counter.
<code>dbg_mtimecmp</code>	<code>uint64_t dbg_mtimecmp(unsigned hart) const</code>	Back-door read of <code>MTIMECMP[hart]</code> .
<code>dbg_msip</code>	<code>uint32_t dbg_msip(unsigned hart) const</code>	Back-door read of <code>MSIP[hart]</code> (0 or 1).
<code>dbg_mtip</code>	<code>bool dbg_mtip(unsigned hart) const</code>	Compute <code>MTIME ≥ MTIMECMP[hart]</code> without touching state.
<code>dbg_set_mtime</code>	<code>void dbg_set_mtime(uint64_t value)</code>	Test-only: force MTIME and trigger a recompute (§12).
<code>dump_state</code>	<code>void dump_state(std::ostream& = std::cout) const</code>	Human-readable snapshot of CLINT state.

3.6 Internal methods and SC processes

Member	Signature	Kind	Purpose
b_transport	void b_transport(tlm::tlm_generic_payload& sc_core::sc_time&)	TLM b_transport callback	Blocking register read/write data path (§4).
transport_dbg	unsigned int transport_dbg(tlm::tlm_generic_payload&)	TLM transport_dbg callback	Side-effect-free back-door register access (§4, §12).
reset_proc	void reset_proc()	SC_METHOD (sensitive to rst_n_i)	Zero MTIME/MSIP, set MTIMECMP=max on assertion; recompute (§9).
tick_method	void tick_method()	SC_METHOD (driven by tick_event_)	Increment MTIME, re-arm next tick, recompute (§8, §10).
output_method	void output_method()	SC_METHOD (driven by recompute_event_)	Sole driver of msip_o[*] / mtip_o[*] (§7).
schedule_recompute	void schedule_recompute()	private helper	Post recompute_event_ at SC_ZERO_TIME (§7).
reg_read	bool reg_read(uint64_t off, unsigned access_size, uint32_t& data_lo, uint32_t& data_hi) const	private helper	Decode a register read; false if out-of- window (§5).
reg_write	bool reg_write(uint64_t off, unsigned access_size, uint32_t data_lo, uint32_t data_hi)	private helper	Decode a register write; false if out-of- window (§5).

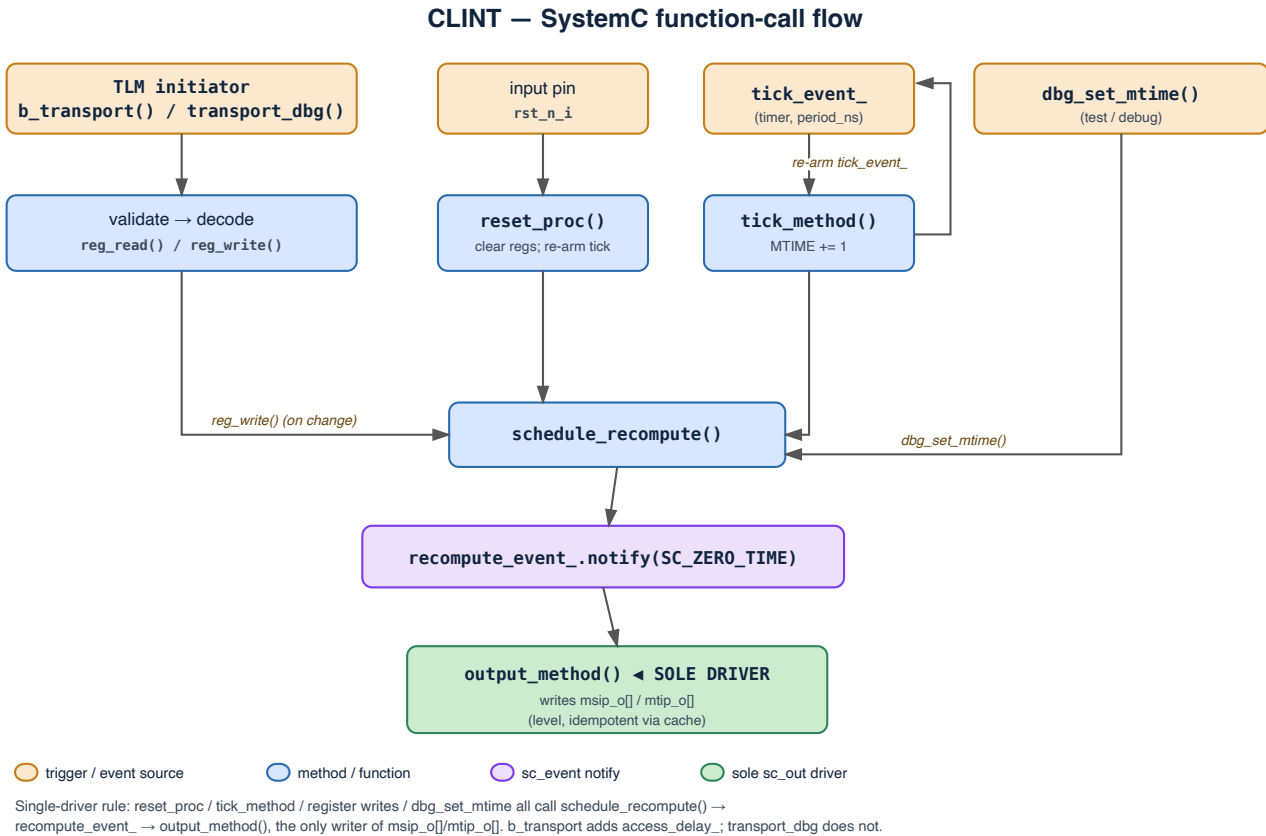
Processes/threads: the model registers exactly **three** SC processes — `reset_proc`, `tick_method`, and `output_method`, all **SC_METHOD**. There are **no** **SC_THREAD** / **SC_CTHREAD** processes. `output_method` is the **only** writer of the `msip_o`/`mtip_o` signals; every other path mutates internal state and calls `schedule_recompute()`, satisfying SystemC 3.0's single-driver rule (§7).

3.7 File-local helpers (clint.cpp anonymous namespace)

Symbol	Definition	Purpose
MSIP_BIT_MASK	constexpr uint32_t MSIP_BIT_MASK = 0x1u	MSIP write mask — only bit[0] is the IPI; bits[31:1] RAZ/WI per <code>clint.rdl</code> .
MTIMECMP_RESET_VALUE	constexpr uint64_t MTIMECMP_RESET_VALUE = UINT64_MAX	Post-reset MTIMECMP so MTIP is deasserted until firmware programs a comparator.

3.8 Function call flow

The diagram below shows how the SC processes, TLM callbacks, and helpers call one another at run time. Every state-changing path converges on `schedule_recompute()`, which fires `recompute_event_` at `SC_ZERO_TIME`; its sole consumer is `output_method()`, the only writer of `msip_o[]` / `mtip_o[]`. The full process / event topology is detailed in §7.



CLINT SystemC function-call flow: register writes, reset_proc, tick_method and dbg_set_mtime all converge on schedule_recompute() then recompute_event_ then output_method() (the sole driver of msip_o[] / mtip_o[]).

Reading the flow:

- Register access** — `b_transport()` validates and decodes via `reg_read()` / `reg_write()`; a write that changes state calls `schedule_recompute()`, and `b_transport()` annotates `delay += access_delay_` (`transport_dbg()` does not).
- Timer** — `tick_event_` runs `tick_method()` (`SC_METHOD`), which does `MTIME += 1`, re-notifies `tick_event_` for the next period, and calls `schedule_recompute()`.
- Reset** — `rst_n_i` runs `reset_proc()` (`SC_METHOD`), which clears the registers, re-arms `tick_event_`, and calls `schedule_recompute()`.
- Debug** — `dbg_set_mtime()` mutates `MTIME` and calls `schedule_recompute()`.
- `schedule_recompute()` → `recompute_event_.notify(SC_ZERO_TIME)` → `output_method()` (sole driver), which writes `msip_o[]` / `mtip_o[]` as levels, idempotently via the per-hart cache.

4. TLM-2.0 interface implementation

4.1 Socket and callbacks

```
reg_socket.register_b_transport (this, &clint::b_transport);
reg_socket.register_transport_dbg(this, &clint::transport_dbg);
```

`b_transport` is the only blocking entry point. The CLINT does not register `nb_transport_*` (TLM AT path is out of scope) and does not register `get_direct_mem_ptr` (DMI is intentionally refused).

4.2 `b_transport` flow

```
b_transport(gp, delay):
    validate length ∈ {4, 8}
    validate alignment(addr, length)
    validate streaming_width == length
    validate byte_enable_ptr == nullptr
    validate addr < WINDOW_SIZE
    if read:
        ok = reg_read(addr, length, lo, hi)
        on success: copy lo (and hi if 8B) into gp.data_ptr
    elif write:
        copy gp.data_ptr into lo (and hi if 8B)
        ok = reg_write(addr, length, lo, hi)
    else:
        gp.set_response_status(TLM_COMMAND_ERROR_RESPONSE)
    if !ok:
        gp.set_response_status(TLM_BURST_ERROR_RESPONSE)
    re-cache access_delay_ from CCI
    delay += access_delay_
    gp.set_response_status(TLM_OK_RESPONSE)
```

4.3 `transport_dbg`

Symmetrical to `b_transport` but with no annotated delay and no error fault propagation: returns 0 for any malformed access (TLM-2.0 convention) and `length` on success. The CLINT has no read side-effects, so `dbg` and `b_transport` are functionally equivalent on reads.

4.4 Why support both 32-bit and 64-bit accesses

The Chipyard RTL routes 64-bit TileLink accesses to MTIME / MTIMECMP (`auto_in_a_bits_data` is 64 bits wide), while the firmware driver in `riscv_clint0.c` always issues two 32-bit MMIO operations (because RV32 hosts cannot natively issue a 64-bit access). Both code paths are exercised in production; the model must accept both.

4.5 Why DMI is refused

The CLINT exposes a 64-bit MTIME counter that mutates *outside* the caller's control (the tick method runs on its own SC_METHOD). DMI would let an observer read partially-updated 64-bit values without atomicity guarantees. Refusing DMI forces the observer through `b_transport`, which always returns a coherent 64-bit snapshot (because reads happen synchronously on the same SC kernel thread that updates MTIME).

5. Register decode

5.1 Single decode entry-point

`reg_read` and `reg_write` share the same four-region cascade. The cost is identical for read and write; we don't share the implementation because the side-effect handling (cache update, recompute scheduling) is naturally different.

5.2 Cascade layout

```
reg_decode(off, size):
    if off >= WINDOW_SIZE                -> false (caller maps to TLM_ADDRESS_ERROR)
    if off ∈ [MSIP_BASE, MSIP_BASE + N·4) -> MSIP[h];      size must be 4
    if off ∈ [MTIMECMP_BASE, MTIMECMP_BASE + N·8) -> MTIMECMP[h]; size 4 or 8
    if off ∈ [MTIME_OFFSET, MTIME_OFFSET + 8) -> MTIME;      size 4 or 8
    otherwise (in-window hole)            -> RAZ/WI; return true with lo=hi=0
```

Where $N = \text{num_harts}$. Each region computes its own per-register index from $(\text{off} - \text{region_base}) / \text{region_stride}$ and per-half offset from $(\text{off} - \text{region_base}) \% \text{region_stride}$.

5.3 64-bit splits

For both MTIME and MTIMECMP, a 4-byte access at `+0` exposes the low half (`bits[31:0]`), and a 4-byte access at `+4` exposes the high half (`bits[63:32]`). The 8-byte form may only be addressed at `+0`; an 8-byte access at `+4` returns false (`TLM_BURST_ERROR_RESPONSE`).

5.4 MSIP write masking

The RDL declares `MSIP.value[0:0]` and `MSIP.rsvd0[31:1] = 0`. The implementation enforces this by:

```
msip_[h] = static_cast<uint8_t>(data_lo & MSIP_BIT_MASK); // MSIP_BIT_MASK = 0x1
```

Reading back returns the same single bit zero-extended into a 32-bit word.

6. Internal data structures

Member	Type	Size	Purpose
<code>mtime_</code>	<code>uint64_t</code>	8 B	Global free-running 64-bit time counter.
<code>mtimecmp_</code>	<code>std::vector<uint64_t></code>	$8 \cdot \text{num_harts}$ B	Per-hart 64-bit comparator.
<code>msip_</code>	<code>std::vector<uint8_t></code>	$1 \cdot \text{num_harts}$ B	Per-hart IPI flag (only bit[0] used).
<code>msip_cache_</code>	<code>std::vector<bool></code>	$\sim 1 \cdot \text{num_harts}$ b	Last-driven <code>msip_o[h]</code> value (idempotence).
<code>mtip_cache_</code>	<code>std::vector<bool></code>	$\sim 1 \cdot \text{num_harts}$ b	Last-driven <code>mtip_o[h]</code> value (idempotence).
<code>recompute_event_</code>	<code>sc_event</code>	1 event	Triggers <code>output_method</code> .
<code>tick_event_</code>	<code>sc_event</code>	1 event	Triggers <code>tick_method</code> .
<code>access_delay_</code>	<code>sc_time</code>	1 time	Cached value of <code>access_delay_ns_p_</code> .
<code>tick_period_</code>	<code>sc_time</code>	1 time	Cached value of <code>tick_period_ns_p_</code> .

Total state for a 4-hart CLINT: **~80 B** (vs ~10 KB for the PLIC at its default 336-source / 8-context configuration). The CLINT is genuinely tiny.

6.1 Cache vectors

`msip_cache_` and `mtip_cache_` track the *last value driven* on the output signals. `output_method` writes a new value only when the computed level differs from the cached value. This:

1. Avoids spurious `value_changed_event` notifications on downstream consumers (which would otherwise wake the CPU model on every recompute even if the level didn't change).
2. Keeps the model's contribution to VCD waveform size bounded — an active-high line with no transition produces no edge in the dump.

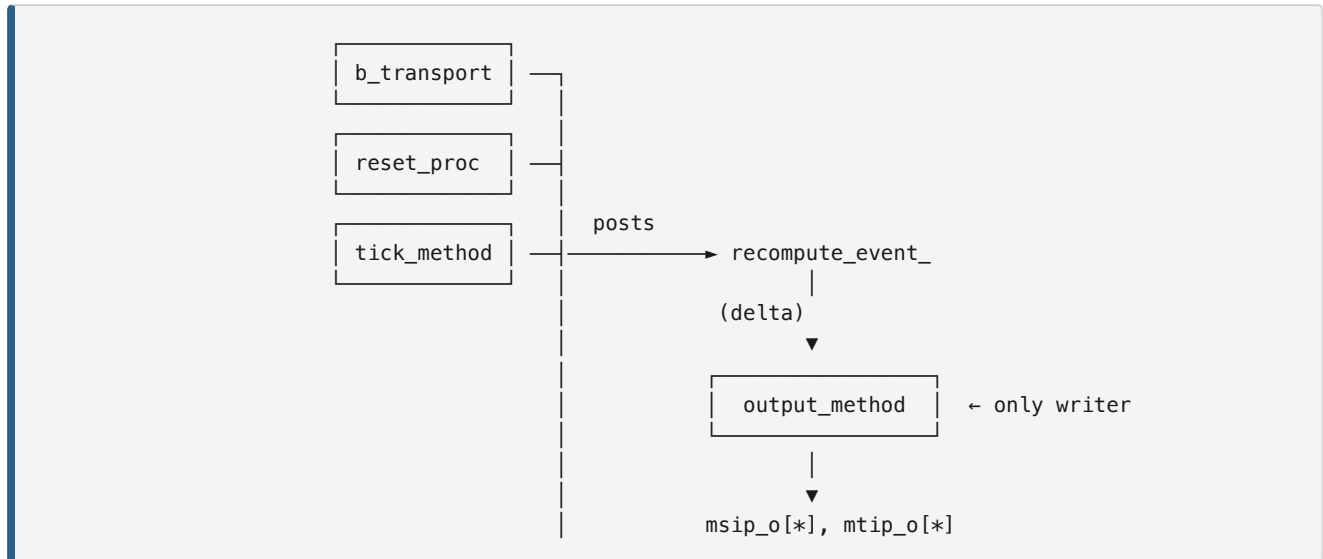
The cache is invalidated by `reset_proc` (which sets all entries to their reset values).

7. Process / event topology — single-driver discipline

SystemC 3.0 enforces a strict single-driver rule: an `sc_signal<T>` may have **exactly one** writer process across its lifetime. The CLINT satisfies this with a three-process layout:

Process	Type	Sensitivity	Writes
<code>reset_proc</code>	<code>SC_METHOD</code>	<code>rst_n_i</code>	internal state, posts <code>recompute_event_</code>
<code>tick_method</code>	<code>SC_METHOD</code>	<code>tick_event_</code>	<code>mtime_</code> , posts <code>recompute_event_</code> , re-arms <code>tick_event_</code>
<code>output_method</code>	<code>SC_METHOD</code>	<code>recompute_event_</code>	<code>msip_o[*]</code> , <code>mtip_o[*]</code> (sole writers)

The TLM `b_transport` callback also mutates internal state and posts `recompute_event_`, but it never directly writes to the output signals — that responsibility is exclusively `output_method`'s.



7.1 `recompute_event_` notification policy

Always notified at `SC_ZERO_TIME` (next delta cycle). Multiple notifications inside the same delta collapse into a single `output_method` invocation — this is correct because `output_method` recomputes the entire output state from scratch, so coalescing is idempotent.

7.2 `tick_event_` notification policy

`tick_event_` is self-arming: `tick_method` schedules the next tick at the end of its body. Reset cancels any pending tick (`tick_event_.cancel()`) and re-schedules from scratch to avoid drift across reset boundaries.

If `tick_period_ns = 0`, `tick_method` is never invoked autonomously and the test bench is responsible for advancing MTIME via writes or `dbg_set_mtime`.

7.3 `dont_initialize()`

All three `SC_METHOD`s call `dont_initialize()` so they do not fire during the elaboration phase. The first real invocation of:

- `reset_proc` happens at the first value-change of `rst_n_i`.
- `tick_method` happens at the constructor-scheduled `tick_event_` (delayed by `tick_period_`).
- `output_method` happens after the first `recompute_event_` posted by any state mutation (TLM write, reset, or tick).

8. Behavioural algorithms

8.1 `output_method`

```
for h in 0 .. num_harts-1:
    msip_lvl = msip[h] & MSIP_BIT_MASK
    mtip_lvl = (mtime_ >= mtimecmp[h])
    if msip_lvl != msip_cache[h]:
        msip_o[h].write(msip_lvl); msip_cache[h] = msip_lvl
    if mtip_lvl != mtip_cache[h]:
        mtip_o[h].write(mtip_lvl); mtip_cache[h] = mtip_lvl
```

Cost: $2 \cdot \text{num_harts}$ comparisons per recompute. For a 4-hart CLINT, 8 comparisons per recompute event.

8.2 `tick_method`

```
if rst_n_i.read() == false: return          # do not tick during reset
mtime_ += 1
schedule_recompute()                        # re-evaluate MTIP for all harts
if tick_period_ != SC_ZERO_TIME:
    tick_event_.notify(tick_period_)        # self-arm next tick
```

Cost: $O(1)$. The actual MTIP recomputation happens in the next delta cycle inside `output_method`.

8.3 `reg_write` for `MTIMECMP` — change detection

Writing the same value twice does not post `recompute_event_`:

```
const uint64_t old = mtimecmp[h];
mtimecmp[h] = ...;           // compute new value
if (old != mtimecmp[h]) changed = true;
if (changed) schedule_recompute();
```

This lets firmware do idle “touch” writes without paying the output-recompute cost. The same pattern is used for MSIP and MTIME.

8.4 32-bit half-word write semantics

A 32-bit write to `MTIMECMP[h]+0` updates only the low half:

```
mtimecmp[h] = (mtimecmp[h] & 0xFFFF'FFFF'0000'0000ULL) | uint64_t(data_lo);
```

A 32-bit write to `MTIMECMP[h]+4` updates only the high half:

```
mtimecmp[h] = (mtimecmp[h] & 0x0000'0000'FFFF'FFFFULL) |
    (uint64_t(data_lo) << 32);
```

Each half-word write atomically updates the relevant 32 bits and posts a single `recompute_event_`. This means `mtip_o[h]` may transiently flip during the firmware’s “high-FF / low / high” sequence, then settle to the final value — exactly matching real hardware behaviour.

9. Reset implementation

```
void clint::reset_proc()
{
    if (rst_n_i.read()) return;          // de-assertion edge: do nothing

    mtime_ = 0;
    fill(mtimecmp_, MTIMECMP_RESET_VALUE); // 0xFFFF...F
    fill(msip_, 0);

    if (tick_period_ != SC_ZERO_TIME) {
        tick_event_.cancel();           // drop any in-flight tick
        tick_event_.notify(tick_period_);
    }

    schedule_recompute();                // output_method drives msip_o / mtip_o low
}
```

Notes:

- The function returns early on **deassertion** so de-asserting reset does not zero MTIME again (which would corrupt the time base). The RTL has the same property — `time_0 <= 0` happens only inside the `if (reset)` branch.
- The MTIMECMP reset value (`0xFFFF...F`) is a **deliberate model choice**: the Chipyard RTL leaves `pad` undefined. We pick max so `mtip_o[h]` is guaranteed deasserted out of reset, eliminating a source of non-determinism. Documented as a deviation in §13 of the spec.
- Cancelling and re-arming `tick_event_` after reset prevents drift: without the cancel, a tick already in the queue would still fire and bump MTIME by 1 just after reset clears it.

10. Loosely-timed timing model

Aspect	Behaviour
Per-access delay	<code>access_delay_ns_p_</code> (default 2 ns) added to each <code>b_transport</code> .
Internal delays	<code>output_method</code> runs in the same delta as the <code>recompute</code> event.
Tick latency	<code>tick_event_.notify(tick_period_)</code> schedules next tick at exactly <code>tick_period_</code> later.
Cross-tick precision	Exactly 1 MTIME unit per tick; no skew, no jitter.
Multi-clock	The CLINT sits on <code>clk_smc_i</code> per tt-oca-hw.pdf §6.6.6 ; in LT this is purely abstract.

`access_delay_ns_p_` is **mutable**: the test bench can change it via the broker between transactions. The implementation re-caches the value at the end of every `b_transport`:

```
access_delay_ = sc_core::sc_time(access_delay_ns_p_.get_value(), SC_NS);
delay += access_delay_;
```

This costs two `double → sc_time` conversions per transaction (cheap) and removes the need for a CCI post-write callback — the model automatically picks up changes on the *next* access.

11. Error handling implementation

Condition	Where caught	Response
<code>length ∉ {4,8}</code>	<code>b_transport</code> validate	<code>TLM_BURST_ERROR_RESPONSE</code>
<code>length=4 ∧ (addr & 3) ≠ 0</code>	<code>b_transport</code> validate	<code>TLM_BURST_ERROR_RESPONSE</code>
<code>length=8 ∧ (addr & 7) ≠ 0</code>	<code>b_transport</code> validate	<code>TLM_BURST_ERROR_RESPONSE</code>
<code>streaming_width ≠ length</code>	<code>b_transport</code> validate	<code>TLM_BURST_ERROR_RESPONSE</code>
Byte-enable supplied	<code>b_transport</code> validate	<code>TLM_BYTE_ENABLE_ERROR_RESPONSE</code>
<code>addr ≥ WINDOW_SIZE</code>	<code>b_transport</code> validate	<code>TLM_ADDRESS_ERROR_RESPONSE</code>
8-byte access on MSIP[h] (4-byte only)	<code>reg_read / reg_write</code>	<code>TLM_BURST_ERROR_RESPONSE</code>
8-byte access on MTIME / MTIMECMP at non-zero half	<code>reg_read / reg_write</code>	<code>TLM_BURST_ERROR_RESPONSE</code>
Unknown command	<code>b_transport</code> switch	<code>TLM_COMMAND_ERROR_RESPONSE</code>
Otherwise	—	<code>TLM_OK_RESPONSE</code>

The unified validation block at the top of `b_transport` keeps the register-decode functions clean — they only return `false` for cases the validator can't see (e.g. wrong access width on a specific register class).

12. Debug & verification hooks

Hook	Purpose
<code>dbg_mtime()</code>	Read MTIME without going through the bus.
<code>dbg_mtimecmp(h)</code>	Read per-hart comparator.
<code>dbg_msip(h)</code>	Read per-hart IPI bit.
<code>dbg_mtipe(h)</code>	Compute MTIP[h] without touching state.
<code>dbg_set_mtime(v)</code>	Force MTIME to <code>v</code> and post recompute. Test-only.
<code>dump_state(os)</code>	Print every register and per-hart line state.
<code>transport_dbg</code>	Standard TLM back-door read/write; symmetrical to <code>b_transport</code> .

`dbg_set_mtime` is the test bench's preferred way to walk MTIME across known MTIMECMP boundaries. Without it, advancing MTIME from 0 to 1 ns before MTIMECMP would require running `tick_period_ns × MTIMECMP - 1` of simulated time — quickly impractical at 64-bit MTIMECMP values.

13. Integration with `smc_top`

```
// In smc_top constructor (4-core variant)
smc::clint clint("clint", smc::clint_cfg{ .num_harts = NCORES });

// Fabric routes BASE+0x0800_0000 .. BASE+0x0800_FFFF here
fabric.to_clint.bind(clint.reg_socket);

// Reset distribution — same source as the PLIC and the cores
clint.rst_n_i(rstu.rst_core_smc_n_o);

// Per-hart interrupt outputs feed the CPU cluster's mip CSRs directly
for (unsigned h = 0; h < NCORES; ++h) {
    clint.msip_o[h](cpu.msip_in[h]);
    clint.mtip_o[h](cpu.mtip_in[h]);
}
```

Inbound `axi_filter` instances belong **between** the fabric and `reg_socket`, exactly as documented for the PLIC in `02_SMC_IP_LowLevel_Design.md §2`.

When the CPU cluster runs Whisper (the SMC programme’s RISC-V ISS of choice, used identically to how the PLIC is integrated), Whisper’s built-in machine-mode timer / SW-interrupt logic must be disabled so all traffic flows through this SystemC model. Whisper loads/stores to the CLINT window are routed via the ISS’s MMIO callback hook into the bus bridge → `reg_socket`, and the model’s `msip_o[h]` / `mtip_o[h]` drive each hart’s `mip.MSIP` / `mip.MTIP` bits through the IRQ aggregator’s call to Whisper’s external-interrupt injection API (typically `Hart::pokeCsr(CsrNumber::MIP, ...)` or `Hart::setMachineSoftwareInterrupt()` / `Hart::setMachineTimerInterrupt()` depending on the Whisper API version). The integration contract is the same shape as a Spike integration; only the C++ symbol names change. See `02_PLIC_LowLevel_Design.md §13.5` for the full Whisper round-trip diagram.

14. Modeling decisions & trade-offs

14.1 Why event-driven MTIME, not a continuous clock

A continuously-clocked counter would generate `tick_period_ns`-aligned wake-ups on every tick **even when nothing else is happening** — a disastrous overhead for firmware idle loops that may span seconds of simulated time with millions of unobserved ticks.

Instead we use a single self-rearming event. The cost per tick is one `SC_METHOD` invocation (≈ 100 ns of host time); the savings come from not having to wake any other process unless a state change actually occurs.

14.2 Why MTIMECMP defaults to all-1s

The Chipyard RTL leaves `pad` (MTIMECMP) without an explicit reset value (`0CAH1CORECluster.CLINT.sv:110`). In silicon, this would be random metastability; in simulation, a default of `0` would assert `mtip_o[h]` continuously out of reset (since $\text{MTIME} = 0 \geq 0$). To make the model deterministic and “do the right thing” without firmware, we default MTIMECMP to `0xFFFF...F`. Firmware must program a real value before unmasking MTIP; this matches best-practice Linux / FreeRTOS boot code anyway.

14.3 Why the cache vectors

Without `msip_cache_` / `mtip_cache_`, every `recompute_event_` would write to every output signal even if the value hadn't changed. The SystemC `sc_signal` machinery does eliminate the resulting `value_changed_event` if the value is identical, but we still pay the write cost and the kernel still inspects it. The cache short-circuits this at the model level.

14.4 Why `cci_param<double>` for `tick_period_ns`

`sc_time` is the natural type but is not a CCI-supported value type. `double` (in nanoseconds) is the next best thing: it serialises to JSON cleanly, is human-readable in inspector tools, and is cheap to convert on demand. The `tick_period_` member caches the converted `sc_time`.

14.5 Why MSIP storage is `uint8_t`

Only bit[0] is meaningful; the other 31 bits are RAZ/WI. Using `uint8_t` saves 3 B per hart and makes the storage type self-documenting (“there is at most 1 valid bit here”). The decode masks with `MSIP_BIT_MASK = 0x1` are therefore strictly defensive — the storage already cannot hold any other bit.

15. Performance & footprint

15.1 State

Configuration	State per CLINT
1 hart	~25 B
4 harts (SMC default)	~80 B
64 harts (max realistic)	~640 B

15.2 Per-event cost

Event	Cost
MTIME tick	$O(1)$ increment + $O(N)$ MTIP recompute via <code>output_method</code> .
MSIP write	$O(1)$ update + $O(N)$ recompute (only the affected bit changes the cache).
MTIMECMP write	$O(1)$ update + $O(N)$ recompute.
MTIME write	$O(1)$ update + $O(N)$ recompute.
Reset assertion	$O(N)$ clear + $O(N)$ recompute.
<code>b_transport</code> baseline	~10 host-cpu ns + 2 sim-ns of annotated delay.

15.3 Tick steady-state cost (4-hart, 10 MHz)

$8 \text{ comparisons} \times 10^7 \text{ ticks/s} \approx 8 \times 10^7 \text{ ops/s}$ of host CPU. Empirically, well under 1 % of a single host core in a Release build. This is small enough that the CLINT is never the bottleneck in any realistic SMC simulation.

16. Known limitations & future work

- **Approximately-Timed (AT) extension:** the LT model collapses every access into a single `b_transport` call. An AT version would split request/response phases and model fabric back-pressure.
- **Parity / lockstep safety extensions:** per `tt-oca-hw.pdf §16` (OCAH-HWSR-SMC-043 et al.), the silicon CLINT implements parity protection on configuration registers and lockstep generation on interrupt outputs. These are functional-safety features not yet modelled here; they would slot in as a wrapper that observes `dbg_*` accessors and raises an injected fault input.
- **DMI for MTIME:** refused today (see §4.5). A DMI-friendly variant could expose MTIME via a small shared-memory window with explicit ordering rules.
- **Variable-rate tick** (e.g. for power-aware SMC modes): would require making `tick_period_ns` mutable and adding a CCI post-write callback to reschedule the in-flight `tick_event_`.

17. Code walkthrough — function reference

Function	File	Purpose
<code>clint::clint(name, cfg)</code>	<code>clint.cpp</code>	Construct CCI params, allocate state, register processes / TLM callbacks.
<code>clint::reset_proc()</code>	<code>clint.cpp</code>	Synchronous reset; clears MTIME / MSIP, parks MTIMECMP at max.
<code>clint::tick_method()</code>	<code>clint.cpp</code>	Increment MTIME, schedule next tick, post recompute.
<code>clint::output_method()</code>	<code>clint.cpp</code>	Sole driver of <code>msip_o[*]</code> / <code>mtip_o[*]</code> .
<code>clint::schedule_recompute()</code>	<code>clint.cpp</code>	Post <code>recompute_event_</code> at <code>SC_ZERO_TIME</code> .
<code>clint::reg_read(off, sz, lo, hi)</code>	<code>clint.cpp</code>	Decode read; populate <code>lo</code> (and <code>hi</code> for 8B).
<code>clint::reg_write(off, sz, lo, hi)</code>	<code>clint.cpp</code>	Decode write; mutate state; post recompute on change.
<code>clint::b_transport(gp, delay)</code>	<code>clint.cpp</code>	Validate <code>gp</code> , call <code>reg_read</code> / <code>reg_write</code> , annotate delay.
<code>clint::transport_dbg(gp)</code>	<code>clint.cpp</code>	Side-effect-free <code>reg_read</code> / <code>reg_write</code> ; no delay.
<code>clint::dbg_mtime / mtimecmp / msip / mtip</code>	<code>clint.cpp</code>	Const accessors for the test bench.
<code>clint::dbg_set_mtime(v)</code>	<code>clint.cpp</code>	Force MTIME and post recompute.
<code>clint::dump_state(os)</code>	<code>clint.cpp</code>	Pretty-print all CLINT state.

18. Build, packaging and dependency notes

18.1 CMake

The library target `smc_clint` exports `include/` as a public include directory and links against `SystemC::systemc` and `SystemC::cci`. Downstream targets need only `target_link_libraries(... PRIVATE smc_clint)`.

The test-bench target `clint_tb` is an `add_executable` that depends on `smc_clint` plus `SystemC::cci`; it is registered with `add_test()` so `ctest` picks it up automatically. The pass criterion is the literal string `ALL TESTS PASSED`; any line containing `FAIL` triggers a test- case failure.

18.2 Discovery

- **SystemC**: prefers a CMake-installed `SystemCLanguage` package; falls back to `SYSTEMC_HOME` if set; otherwise fails. The `run_tests.sh` helper auto-probes `/opt/homebrew/opt/systemc`, `/usr/local/opt/systemc`, `/usr/local`, and `/usr` for a SystemC install.
- **CCI**: located via `CCI_HOME` (default `/Users/pdroy/ccl`). The helper script also probes `/usr/local/ccl` and `/opt/homebrew/opt/systemc-ccl`. On macOS, `DYLD_LIBRARY_PATH` is extended so the dynamic CCI libraries are resolvable at run time.

18.3 Instrumented variants

`run_tests.sh --asan` builds in `build_asan/` with AddressSanitizer enabled and runs the test bench under `ASAN_OPTIONS=halt_on_error=0`. `run_tests.sh --coverage` builds in `build_cov/` (Debug + LLVM source-based or gcov coverage), runs the binary, and emits a per-line report plus an HTML coverage tree.

18.4 Out-of-source builds

All build trees live under `peripherals/clint/build*` and are gitignored / cleanable. Headers and sources never know their build directory; CMake's `target_include_directories(... PUBLIC include/)` is the single source of truth for the include path.